

GS_PINO project guide

This document describes the intended workflow, data layout, neural operator, and evaluation outputs for the fixed-boundary Grad-Shafranov masked U-FNO/PINO surrogate.

1. Goal

The model learns a parameterized solver surrogate

```
[R0, a, kappa, delta, Ip, betap, alpha_m, alpha_n] + (R, Z, LCFS geometry)
-> normalized psi_bar(R, Z)
```

Only points inside the LCFS are physically meaningful. The implementation keeps a rectangular tensor grid for U-FNO efficiency and uses LCFS masks/signed-distance features to isolate the non-rectangular plasma domain.

2. Directory structure

configs/default.yaml	Small development configuration.
configs/practical.yaml	Larger practical experiment configuration.
scripts/run_smoke.sh	Tiny generate/train/evaluate smoke workflow.
scripts/run_practical.sh	Larger end-to-end workflow for usable experiments.
src/gs_pino/geometry.py	LCFS, rho, theta, mask, and signed-distance helpers.
src/gs_pino/solvers.py	Solver adapter location; currently analytic fallback.
src/gs_pino/generate_dataset.py	Dataset generation CLI.
src/gs_pino/data.py	Dataset loader and U-FNO input-channel construction.
src/gs_pino/models.py	Spectral convolution, U-Net branch, and UFN02d.
src/gs_pino/losses.py	Masked supervised and PINO-style regularizer losses.
src/gs_pino/train.py	Training CLI and checkpoint writer.
src/gs_pino/evaluate.py	Metrics and visualization CLI.

3. Dataset structure

`python -m gs_pino.generate_dataset` writes a compressed `.npz` file. Every array uses sample-major layout. For `N` samples and an `nr` x `nz` grid:

Key	Shape	Meaning
R	[N, nr, nz]	Major-radius coordinate grid for each case.
Z	[N, nr, nz]	Vertical coordinate grid for each case.
params	[N, 8]	Ordered parameters: R0, a, kappa, delta, Ip, betap, alpha_m, alpha_n .
psi	[N, nr, nz]	Physical-scale flux from the solver/fallback.
psi_bar	[N, nr, nz]	Normalized target flux; LCFS is approximately zero and the axis is one.
mask	[N, nr, nz]	1 inside LCFS, 0 outside LCFS.
sdf	[N, nr, nz]	Approximate signed distance proxy; negative inside, zero near LCFS.
rho	[N, nr, nz]	Miller-like normalized radius; LCFS is $\rho = 1$.
theta	[N, nr, nz]	Miller-like poloidal angle.
axes	[N, 4]	R_axis, Z_axis, psi_lcfs, psi_axis .
param_names	[8]	Names matching the parameter order.

The current `AnalyticFixedBoundarySolver` is a development adapter. For real physics data, replace its `solve()` method with the external `GS_solver` call and return the same fields: `psi` , `psi_bar` , `psi_lcfs` , `psi_axis` , `R_axis` , and `Z_axis` .

4. Input channels used by U-FNO

`GSDataset` builds a channel-first tensor `[c, nr, nz]` for each sample:

1. $(R - R_0) / a$
2. Z / a

3. $z / (\kappa * a)$
4. `inside_mask`
5. `sdf`
6. `rho`
7. `sin(theta)`
8. `cos(theta)`
9. normalized r_0 , broadcast to the grid
10. normalized a , broadcast to the grid
11. normalized κ , broadcast to the grid
12. normalized δ , broadcast to the grid
13. normalized I_p , broadcast to the grid
14. normalized β , broadcast to the grid
15. normalized α_m , broadcast to the grid
16. normalized α_n , broadcast to the grid

The non-rectangular LCFS is therefore represented by input features and by masked losses, while the neural operator still runs on a rectangular tensor.

5. Model structure

UFNO2d is a compact U-FNO-style model:

1. **Lifting layer:** 1×1 Conv2d($C \rightarrow \text{width}$) maps input channels into a latent field.
2. **Repeated UFNO blocks:**
 - spectral convolution in Fourier space for global operator-like mixing;
 - pointwise 1×1 convolution for local channel mixing;
 - a small U-Net branch for non-periodic/local spatial corrections;
 - GELU activation.
3. **Projection head:** 1×1 Conv2d($\text{width} \rightarrow 128 \rightarrow 1$) returns $\bar{\psi}$.

The model predicts values over the full rectangular grid, but only the in-LCFS portion is used for the main metrics and supervised loss.

6. Losses

Training combines:

```
loss = masked_data_mse
      + bc_weight * boundary_band_loss
      + pde_weight * interior_elliptic_regularizer
```

- `masked_data_mse` uses only `mask == 1` points.
- `boundary_band_loss` pushes `psi_bar` toward zero near `sdf == 0`.
- `gs_residual_loss` is currently a finite-difference elliptic smoothness proxy. It should be replaced by the exact Grad-Shafranov residual once the exact $p'(\psi)$ and $FF'(\psi)$ parameterization from the external solver is wired in.

7. Evaluation outputs

`python -m gs_pino.evaluate` writes:

```
metrics.json
summary_error_histogram.png
summary_error_vs_parameters.png
cases/case_000.png
cases/case_001.png
...
```

Each per-case plot contains three panels: solver truth, model prediction, and error. The title includes all eight input parameters so bad cases can be traced back to the equilibrium settings. The aggregate plots show overall test-set error distribution and error versus each of the eight input parameters.

8. Recommended workflows

Smoke test

Use this to check code paths on a small machine:

```
./scripts/run_smoke.sh
```

Practical run

Use this after installing dependencies and connecting the real GS solver:

```
./scripts/run_practical.sh
```

You can override the larger script without editing it:

```
N_SAMPLES=8192 NR=192 NZ=192 EPOCHS=400 BATCH_SIZE=4 ./scripts/run_practical.sh
```

9. Next physics integration steps

1. Replace the analytic fallback with the real fixed-boundary `GS_solver` call.
2. Save exact solver metadata: `psi_lcfs` , `psi_axis` , `R_axis` , `Z_axis` , and the profile normalization used for `Ip` / `betap` .
3. Replace the placeholder elliptic regularizer with the exact GS residual.
4. Add integral constraints for `Ip` and possibly `betap` once current and pressure definitions are available from the solver.